BaiZe Technical Report

***Abstract*—Large language models (LLMs) exhibit strong reasoning and tool-use skills, yet their computational footprint makes most agentic models impractical for edge, embedded, and cost-sensitive electronic design automation (EDA) deployments. We present BaiZe, a ≈2.2-billion-parameter agentic language model built *from scratch* on a fixed compute budget around a *hybrid* state-space backbone. BaiZe adopts a sequential hybrid architecture—56 layers interleaving Mamba-2 state-space layers, four grouped-query-attention layers, and MLP layers—whose linear-time decoding is well suited to inference-heavy, cost-sensitive agents, and is trained with the NVIDIA NeMo recipe on Megatron-Core. The architecture is chosen through a matched from-scratch comparison against a dense transformer counterpart, in which the hybrid attains lower loss and faster inference with fewer parameters at a modest training-throughput cost. We then run a staged, budget-constrained hyper-parameter search (S1–S5) on $8\times$H100 over the final hybrid backbone, which selects a stable learning rate of 1×10^{-3} under a warmup–stable–decay schedule with an $86{:}10{:}4$ annealing mixture, reaching a validation loss of $\mathbf{2.2054}$ after 20000 steps and 2.6739 ± 0.0469 across three seeds at 5000 steps. The search recipe is then scaled to a full production run of ≈44B tokens—the Chinchilla-optimal budget for a 2.22B model—reaching a final training loss of $\mathbf{1.357}$ in ≈412 GPU-hours on $8\times$H100. We release the complete recipe and training command to support the reproducible construction of efficient agentic small language models.**

***Index Terms*—small language model, agentic LLM, hybrid architecture, state-space model, budget-constrained search, efficient inference**

I. Introduction

Large language models have recently demonstrated agentic capabilities—planning, tool invocation, and long-horizon reasoning—that make them attractive cores for intelligent assistants spanning chip design, verification, and other domains of electronic design automation. However, state-of-the-art agent backbones are routinely 7 B parameters or larger, demanding high-end accelerators and memory footprints that are incompatible with edge, embedded, and cost-sensitive deployments where inference latency and energy are first-class constraints. The constraint is sharper than raw cost in the settings we target: proprietary design artifacts—layouts, schematics and review judgements—cannot leave the premises, so an agentic core must be served locally; and because an agentic session accumulates context, what dominates serving cost is not prompt throughput but the memory that grows with the session. For a hybrid backbone this is architectural: only the four attention layers carry a session-growing state, while the Mamba-2 layers hold a constant-size recurrent one.

Recent small language models (SLMs, < 2 B) have closed much of the gap with larger counterparts on single-turn benchmarks, but building such a model *from scratch* remains an under-documented engineering problem. Every choice—architecture, optimizer, learning-rate schedule, data mixture—interacts with every other, and a naive grid search is prohibitively expensive. Our goal is therefore twofold: (i) to construct, from scratch and on a strictly limited compute budget, an end-to-end multimodal agentic system—an efficient hybrid language core paired with a custom vision encoder, aligned into a semiconductor-domain agent—and (ii) to expose the concrete, reproducible decisions that lead to it.

a) BaiZe.: We present BaiZe, an end-to-end multimodal agentic system assembled from scratch across five stages: (i) pre-training a ≈2.2B-parameter hybrid (state-space + attention) LLM, (ii) post-training it for agentic capability, (iii) pre-training a custom vision encoder, (iv) aligning the two into a multimodal model, and (v) adapting the result to the semiconductor domain. Three design pillars organise our work:

- **Architecture.** A sequential hybrid language backbone—56 layers interleaving Mamba-2 state-space layers, four grouped-query-attention layers, and MLP layers—giving linear-time decoding for inference-heavy agents, paired with a from-scratch vision encoder.
- **Reduced-horizon proxy search.** We exploit short, cheap proxy runs to rank candidate hyper-parameters, so that the ranking discovered at small scale carries over to the full model without re-tuning at scale.
- **Budget-constrained stage-wise pipeline.** Each of the five stages is trained under a hard GPU-hour budget, with a staged search protocol (S1–S5) for the language backbone and a single frozen evaluation protocol shared by the later stages.

We hope BaiZe serves as a minimal, reproducible blueprint for building efficient agentic SLMs from scratch, and that our search methodology is of practical value to cost-bounded training efforts in the EDA and embedded-AI communities.

II. Background and Related Work

A. Small Language Models and Agentic Reasoning

Parameter-efficient SLMs (< 2 B) have recently narrowed the gap with larger models on mathematical and commonsense reasoning. MiniCPM [1] introduced scalable training strategies and domain-weighted data mixing for small models; Qwen [2] and Gemma [3] demonstrate that strong code- and math-enriched corpora can lift small models substantially. Most of these efforts, however, treat the final trained model as the deliverable and report comparatively little about the *search* that produced it. Complementarily, agentic instruction-tuning datasets (e.g., tool-use and multi-step reasoning corpora) [4] have been shown to elicit planning and tool-invocation behaviour, yet are rarely paired with a from-scratch pre-training recipe for the 1B regime.

B. Hybrid and State-Space Architectures

State-space models (SSMs) such as Mamba [5] and Mamba-2 [6] replace full attention with a recurrent scan update, offering linear-time decoding that suits inference-bound deployments. Hybrid designs interleave a small number of attention layers among SSM/MLP layers—notably Nemotron-H [7] and Hymba [8]—to recover long-range context while retaining fast decode. Released Megatron-Core variants of these families are scarce below 2B, leaving their small-scale trade-offs underdocumented; our controlled 2B comparison (Section IV-A) addresses precisely this gap.

C. Multimodal Vision-Language Models

Vision-language models connect a vision encoder to a language model through a lightweight projector, enabling end-to-end visual understanding and instruction following. LLaVA [9] established visual instruction tuning—aligning a vision encoder to a frozen LLM via a linear projection followed by instruction fine-tuning—as a scalable recipe for open multimodal assistants. LLaVA-OneVision [10] consolidates this line into a single open model spanning single-image, multi-image, and video tasks through a staged curriculum: stage-1 projector pretraining, stage-1.5 mid-training, and stage-2 instruction tuning. BaiZe follows this curriculum in stage (iv) (Section VII), substituting a from-scratch vision encoder (stage iii) for the frozen off-the-shelf encoder.

D. LLMs for Electronic Design Automation

A growing line of work applies LLM agents to chip design tasks such as register-transfer-level generation, verification, and bug triage [11], [12]. These applications motivate a cheap, always-on, agentic multimodal core that can run per-team or per-engineer; our work supplies the efficient multimodal backbone such systems require.

III. Architecture and Pipeline Overview

BaiZe is an end-to-end multimodal agentic system assembled in five stages: (i) from-scratch pre-training of a hybrid language backbone, (ii) post-training of that backbone (instruction tuning and reinforcement learning to strengthen agentic capability), (iii) pre-training of a custom vision encoder, (iv) alignment of the two into a multimodal large language model, and (v) semiconductor-domain adaptation via reinforcement learning. This section fixes the shared scaffolding—the two backbones, the overall pipeline, and the global training environment—before each stage is detailed in its own section.

A. Language Backbone

The language backbone is a sequential hybrid model [6], [7]: 56 transformer layers interleave Mamba-2 state-space layers, four grouped-query-attention (GQA) layers, and MLP layers. The layer allocation follows the Nemotron-H sequential hybrid convention—four attention layers among the 56, the remainder Mamba-2 and MLP—as provided by Megatron-Core's Mamba-hybrid recipe, whose 56-layer pattern we reuse verbatim while halving only the hidden and FFN widths to reach the ≈2B scale. BaiZe is not a Nemotron-H model: it is an order of magnitude smaller, contains no mixture-of-experts layers, and is pre-trained from scratch on an independent corpus under a fixed GPU-hour budget. Each Mamba-2 layer uses 8 state-space groups; the four attention layers use 16 query heads and 4 query groups (GQA) with untied word embeddings. Crucially, only the four attention layers carry rotary position embeddings (RoPE); the 52 Mamba-2 state-space layers are position-agnostic, so the backbone's long-context extrapolation quality depends on just $4/56{=}7.1\%$ of layers, an asymmetry exploited in Section IV-L. The full configuration is listed in Table I; the model totals ≈2.22B parameters.

TABLE I
BaiZe language-backbone configuration (Mamba2-hybrid).

Hyper-parameter	Value
Transformer layers	56
Architecture	hybrid (Mamba-2 + GQA + MLP)
Attention layers	4 (interleaved)
Hidden size	2048
FFN hidden size	8192
Attention heads (Q)	16
Query groups (GQA)	4
Mamba-2 state-space groups	8
Vocabulary size	129,281
Sequence length	4096
Activation	SiLU / SwiGLU
Normalisation	RMSNorm
Precision	BF16
Parameters	≈2.22B

TABLE II
BaiZe five-stage pipeline overview.

Stage	Task	Status	Section
(i)	LM pre-training	complete	IV
(ii)	Post-training (SFT, RLVR)	design	V
(iii)	Vision encoder	complete	VI
(iv)	MLLM alignment	design	VII
(v)	Domain adapt. (RLHF)	design	VIII

B. Vision Encoder

The vision encoder is pre-trained from scratch in stage (iii) and is selected from six iso-parameter candidates at the 500–600 M scale; Section VI reports the comparison and its outcome.

C. Pipeline and Global Environment

Table II summarises the five-stage pipeline. All stages run on 8×NVIDIA H100 GPUs (CUDA 12.8, PyTorch 2.8.0, Python 3.10); fused kernels, selective activation recomputation, and cross-entropy loss fusion are enabled where applicable, with `tensor_model_parallel_size = 1` and BF16 precision throughout.

IV. LLM Pretraining

Stage (i) pre-trains the hybrid language backbone (Section III-A) from scratch under a fixed budget. This section

covers the architecture-selection rationale, the data, the production recipe—organised as a warmup–stable–decay (WSD) schedule whose decay phase carries two segments, a *short-context annealing segment* and a *long-context extension segment* (Section IV-K)—and the resulting measurements; the backbone configuration itself is given in Table I.

A. Architecture Selection Rationale

We adopt the hybrid backbone on three grounds. **(i) Inference efficiency.** State-space layers decode with constant-time recurrence per token, while the sparse attention layers retain long-range context; this directly targets the inference-heavy, cost-sensitive deployments that motivate BaiZe. **(ii) Controlled comparison.** A matched from-scratch comparison at the ≈2B scale between a dense transformer (MiniCPM5-2B [13]) and Mamba2-hybrid—trained under identical data, seed, and GPU footprint—favours the hybrid, which converges to a lower loss with fewer parameters (≈2.22B vs. 2.51B, -11.6%) and decodes faster under our direct-drive harness, at a $\sim$8% training-throughput cost (Table VII). Across all five checkpoints from 500 to 5000 steps, the hybrid consistently attains lower loss than the dense baseline, with the gap widening to $\Delta\approx 0.30$ by step 5000—confirming the advantage is not an early-training artifact and comes with an 11.6% parameter reduction. **(iii) Feasibility.** Megatron-Core ships a Mamba-hybrid provider whose 56-layer interleaving pattern we reuse verbatim, halving only the hidden and FFN widths to reach the ≈2B scale, making the backbone directly executable under the NeMo recipe without custom kernels.

B. Training Framework and Reduced-Horizon Search

We train with the NVIDIA NeMo [14] pretraining entry point, driving the Megatron-Core [15] `NVIDIAMambaHybridModelProvider2B` provider directly with `tensor_model_parallel_size` $= 1$. To keep the entire sweep within budget, we run all hyperparameter searches at a reduced iteration count and treat the resulting short-horizon ranking as predictive of the full-model optimum. Fused kernels, selective activation recomputation, and cross-entropy loss fusion are enabled to raise throughput.

C. Learning-Rate Schedule: WSD with a Two-Segment Decay

BaiZe trains under a warmup–stable–decay (WSD) schedule with a stable learning rate of 1×10^{-3} and 2,000 warmup steps (Table III). The schedule has three segments, and the organisation of the third is the subject of Section IV-K:

- **Warmup + stable** (700,000 steps, 4.19M tokens/step, 2.94T tokens; Table III). The learning rate is held at its peak. The main training sequence length is 16,384; this is a deliberate choice, not a limitation: 16K is the throughput-optimal operating point of the backbone on our cluster (613,992 tok/s at `CP`=1/`DP`=16 with computing-graph replay off; measured), and it keeps the extrapolation ratio to the 1M product target at $64\times$ instead of the $256\times$ that a 4K pretraining length would imply.
- **Decay—segment A, short-context annealing** (60–70% of the decay budget). The learning rate enters its decay while the sequence length stays at 16K, annealing the model on the high-quality instruction and curated-web mixture.
- **Decay—segment B, long-context extension** (30–40% of the decay budget). The sequence length is extended on a 32K→64K→128K ladder while the decayed learning rate continues downward (Section IV-K).

The decay segment itself follows $lr(p) = lr_{\text{peak}}\,(lr_{\min}/lr_{\text{peak}})^{p}$ with $p \in [0,1]$ (exponential) and $lr_{\min}{=}1 \times 10^{-4}$, the setting retained by the schedule-family ablation of Section IV-I; the per-step values it produces ($p{=}0.25/0.5/0.75 \Rightarrow 5.62/3.16/1.78 \times 10^{-4}$) are the learning rates under which segment B runs.

TABLE III
PRODUCTION WSD BUDGET. TOKENS PER STEP ARE NOMINAL (`GBS` × `SEQ`); WALL-CLOCK AND GPU-HOURS ARE ESTIMATES AT THE MEASURED STABLE-PHASE THROUGHPUT. DECAY IS BUDGETED AS *at least* 10% OF THE STABLE PHASE (TABLE X).

Segment	Tok./step	Steps	Tokens	Seq.
Warmup + stable	4.19M	700,000	2.94T	16K
Decay A: short-ctx anneal	4.19M	14,000	58.7B	16K
Decay B: long-ctx ladder	4.19M	6,000	25.2B	32K–128K
Total	—	**720,000**	**3.02T**	—

The decay row is the *reference allocation* used throughout this paper (Section IV-K discusses scaling it upward to the 10–15% range of Table X). It inherits the production invariant that every rung keeps 16,384 tokens per micro-batch per GPU (`seq/CP` $= 16{,}384$ with `CP` $\in \{1,2,4,8\}$), so no rung exceeds the memory ceiling established for the stable phase.

D. Search Protocol Preceding the Production Run

Because a full grid is infeasible under a fixed GPU budget, the production recipe was selected with a staged, single-variable-at-a-time protocol run on the same backbone at a reduced horizon:

- **S1 (baseline).** Reproduce the Mamba2-hybrid reference configuration (stable LR 3×10^{-4}, WSD) to establish a loss/throughput baseline.
- **S2 (learning-rate sweep).** Sweep the stable learning rate over seven points from 2×10^{-4} to 1×10^{-3} with WSD held fixed.
- **S3 (schedule axes).** On the winning learning rate, perturb one variable at a time—schedule family (WSD vs. cosine), decay ratio and decay shape, warmup length, and min_lr.
- **S4 (annealing data mix).** Ablate the annealing data blend (pure Ultra-FineWeb-L3 vs. adding code and math) under the winning configuration.
- **S5 (long run + seeds).** Retrain the winning configuration from scratch over a 20000-step horizon, plus a three-seed short-horizon reproducibility check.

These sweeps are *proxy experiments*: their hyper-parameter ranking is transferred to the production scale, and the absolute losses they report are not production estimates. The production run is specified by the budget of Table III and is reported in Section IV-J; the long-context segment of that budget is specified in Section IV-K.

At every stage only a single variable changes relative to the running best, isolating each factor's contribution while keeping total GPU-hours within budget. The architectural search space is validated up front: the hybrid (Mamba2-hybrid) and dense (MiniCPM5-2B) candidates are compared head-to-head from scratch (Section IV-A), which selects the hybrid backbone before the hyper-parameter search begins.

E. Data

Pre-training uses the `Ultra-FineWeb-L3` English subset (≈690 B tokens across 616 parquet shards, 618 GiB), streamed and packed, tokenised with the DeepSeek-V4.1-Flash tokenizer [16] (vocabulary size 129,281). Complementary corpora—code (`UltraData-Code`), mathematics (`UltraData-Math`), curated web (`Ultra-FineWeb` tiers), and instruction-tuning sets (`UltraData-SFT-2605`, `UltraData-SFT-Agent-2609`)—supply the production blend and the two decay segments. The stable phase draws on the seven-source mixture of Table IV, which keeps the 88:8:4 web/code/math prior selected in Section IV-G at the top level while promoting high-quality tiers (L1/L2) into the stable phase and reserving the L3 tiers for annealing. Sources are pre-blended into large shards before the run, so that each rank opens $\mathcal{O}(10)$ index files rather than the $\mathcal{O}(600)$ of the raw layout; this keeps the data-loader resident set bounded and is a precondition for the throughput reported in Section IV-J.

TABLE IV
STABLE-PHASE DATA MIXTURE (SUMS TO 1.000; "EPOCHS" IS THE EXPECTED NUMBER OF PASSES OVER THE SOURCE AT 2.94T TOKENS). THE BLEND FOLLOWS THE 88:8:4 WEB/CODE/MATH PRIOR OF SECTION IV-G.

Source	Description	Prop.	Epoch.
`en_v1_4`	`Ultra-FineWeb` EN, filtered v1.4	0.65	0.81
`ufw_l1_en_hq`	`Ultra-FineWeb` EN, high-quality tier	0.05	0.93
`ufw_zh`	`Ultra-FineWeb` ZH	0.04	0.99
`math_l1`	`UltraData-Math`, tier L1	0.05	0.85
`math_l3`	`UltraData-Math`, tier L3	0.02	0.72
`code_l2`	`UltraData-Code`, tier L2	0.15	1.00
`code_l3`	`UltraData-Code`, tier L3	0.04	0.87

Contamination control is enforced before any corpus enters training: a blacklist of 193,295 character-level 13-gram fingerprints (FNV-1a 64-bit hashed, NFKC-normalised) is built from all 536 tasks across six evaluation snapshots of EDA-Eval-PyAether, and every new data shard—including SFT corpora—is scanned for ≥ 0.80 n-gram overlap before release (Section V-D).

F. Long-Context Data Prerequisites

Segment B learns from whatever length structure the corpus actually contains, so before budgeting it we measured the document-length distribution of the main source with an EOD-aware scan (`tokenizer_eod` = 129,280; 8 shards, 8.59B tokens, 9.66M documents). The result is the binding constraint on Section IV-K: only $\mathbf{2.52}\%$ of tokens sit in documents of $\geq$32K, only 0.03% reach $\geq$128K, and the mean document is 889 tokens (Table V). Training a 128K rung on this distribution would teach *document concatenation* rather than long-context retrieval: the model would see 128K windows whose content is a chain of unrelated short documents, and its long-context score would be inflated relative to any real long document.

TABLE V
MEASURED DOCUMENT-LENGTH DISTRIBUTION OF THE MAIN PRE-TRAINING SOURCE (`EN_V1_4`; 8 SHARDS, 8.59B TOKENS, 9.66M DOCUMENTS, MEAN 889 TOKENS/DOC, MAX 195,935). "TARGET" IS THE SEGMENT B ENTRY GATE; STATUS IS MEASURED.

Threshold	Documents	Token share	Status vs. target
$\geq$32K	4,257 (0.04%)	**2.52%**	target $\geq$20%: **not met**
$\geq$128K	—	**0.03%**	target $\geq$5%: **not met**
$\geq$1M	$< 0.01\%$	not estimated	not required (extrapolated)

We therefore treat the following as *entry conditions* for segment B, not as tuning knobs. **(a) Long-document supplementation.** Books, scientific articles, legal and standards text, and repository-level code (multi-file, in repository-consistent order) are added until the $\geq$32K token share reaches $\geq 20\%$ and the $\geq$128K share reaches $\geq 5\%$; until then the ladder stops at 64K. Length-weighted sampling is applied as well, but it cannot substitute for supply: re-weighting the present corpus raises the $\geq$32K share only by repeating a small number of documents. **(b) Document-level reset.** Packed shards concatenate documents back-to-back, so segment B requires the attention and Mamba-2 recurrent state to be reset at every EOD; without it a 128K window mixes documents and the position statistics the retrieval behaviour depends on are wrong. **(c) Long-document de-duplication.** Near-duplicate long documents are removed first, because a 128K rung consumes few optimisation steps relative to its token budget and duplication is amplified by length. Item (b) is a framework capability rather than a data transformation; we list it here because it gates the same decision.

G. Data Mixture Proportion Search

Because the optimal web/code/math ratio is scale-dependent and a full-scale sweep is infeasible, we searched the three-way mixture at a proxy scale using Bayesian optimisation. A compact proxy model (d=128, L=14, ≈18.36 M parameters—two orders of magnitude smaller than the 2.2 B target and not intended as a performance predictor) was trained for each trial under Optuna's `TPESampler` with `MedianPruner`, one trial per GPU, global batch size 2048 at sequence length 4096. The objective was the average accuracy on eight common-sense `lm-eval` tasks (`arc_challenge`, `arc_easy`, `boolq`, `hellaswag`, `openbookqa`, `piqa`, `sciq`, `winogrande`), each evaluated with `--limit 500` subsampling.

Two rounds of 200 trials each were completed. In the first round, using final training loss as the objective, the Spearman rank correlation between loss-ranked and accuracy-ranked mixtures was $\rho{=}-0.43$, confirming that loss is a weak proxy for downstream ability. In the second round, using the eight-task average directly, the top-5 mixtures were retrained and re-evaluated at the full `lm-eval` scale. Table VI reports the results: the full-eval rankings were nearly inverted relative to the Bayesian-optimisation rankings (Spearman $\rho{=}-0.80$, $n{=}5$, $p{\approx}0.10$), and the spread across all five mixtures was only 0.383–0.390 (≈ 0.6 pp).

TABLE VI
TOP-5 DATA MIXTURES FROM THE SECOND BAYESIAN-OPTIMISATION ROUND, RETRAINED AND EVALUATED AT FULL `LM-EVAL` SCALE. THE BO RANKING (COLUMN 5) IS NEARLY INVERTED RELATIVE TO THE FULL-EVAL RANKING (COLUMN 6), AND THE SCORE SPREAD IS ≈0.6 PP.

Trial	Web	Code	Math	BO	Full
23	0.941	0.109	0.010	**0.4155**	0.3846
38	0.895	0.100	0.010	0.4138	0.3835
118	0.874	0.071	0.055	0.4130	0.3877
199	0.931	0.098	0.010	0.4123	**0.3899**
33	0.940	0.086	0.010	0.4090	0.3897

At the proxy scale of 18.36 M parameters, data-mixture rankings were indistinguishable within 0.6 pp, leading us to adopt the 88:8:4 (web/code/math) prior for the *stable* phase, rather than over-fitting to proxy-scale noise. The empirically optimal region was stable at web 0.87–0.94, code 0.06–0.12, math ≈0.01, consistent with the prior. Raising the discrimination floor—via a larger proxy, multiple seeds, or full-scale evaluation replacing the `--limit 500` subsampling—is left to future work.

H. Search Budget

The pre-training search is bound by a ≈92 GPU-hour plan on eight H100 GPUs (≈192 GPU-hours wall-clock available); ≈85 GPU-hours were consumed. All short-horizon sweeps (S1–S4) run 5000 steps at a global batch size of $8 \times 4096 \approx 32.8$K tokens/step (≈164M tokens, ≈4.0 GPU-hours each); the verification long run (S5) runs 20000 steps (≈655M tokens, ≈13.9 GPU-hours). These token budgets are far below the Chinchilla-optimal scale (≈44B tokens for a 2.22B model); the search is therefore a *proxy experiment*—its hyper-parameter ranking is treated as predictive of the full-scale optimum, not as a converged performance estimate. Validation loss is reported at matched step counts so that comparisons across configurations remain apples-to-apples.

I. Results of the Search

The search selects a stable learning rate of 1×10^{-3} under a warmup–stable–decay (WSD) schedule (warmup 250 steps, decay 500 steps, $min_lr{=}1 \times 10^{-5}$) with an 86:10:4 blend of Ultra-FineWeb-L3, code, and math data in the annealing tail. This is the recipe carried into the production run (Section IV-J), where only the token budget and the sequence ladder are scaled up. Over the 5000-step horizon the validation loss reaches $\mathbf{2.7647}$. The learning-rate sweep improves monotonically from 2.9037 ($2{\times}10^{-4}$) to 2.7627 ($1{\times}10^{-3}$); because $1{\times}10^{-3}$ is also the grid maximum, that on its own leaves open whether the true optimum lies beyond the grid. Extending the grid upward closes the question: 1.5×10^{-3} gives 2.7752 and $2{\times}10^{-3}$ gives 2.7889, so the sweep is *U-shaped* and $1{\times}10^{-3}$ is an interior minimum rather than a clipped boundary. Each short-horizon ablation confirms a single-variable choice: WSD beats cosine (2.7679 vs. 2.8155), a 10% decay ratio beats 5% (2.7679 vs. 2.7900), a 250-step (5%) warmup beats a fixed 2000-step warmup (2.7679 vs. 2.7957), and $min_lr{=}1{\times}10^{-5}$ and 3×10^{-5} are statistically indistinguishable (2.7647 vs. 2.7679, $\Delta{\approx}0.003$ within run-to-run noise of $\sigma{\approx}0.035$ across $n{=}5$ seeds); we adopt $min_lr{=}1 \times 10^{-5}$ as default. Adding code and math to the annealing mix lowers the final loss monotonically (2.7621 pure L3 → 2.6793 L3+code → 2.6298 L3+code+math).

The 20000-step verification run reaches a final training loss of $\mathbf{2.2054}$, with the decay tail (steps 19500–20000) annealing the loss from $\sim$ 2.34 to 2.2054 (annealing gain $\sim$ 0.14; Figure 1). Across three seeds the 5000-step loss is 2.6739 ± 0.0469 (mean $\pm$ sample standard deviation, $n{=}3$; per-seed 2.6402, 2.6539, 2.7275).

Table VII contrasts the hybrid backbone against a dense transformer at the ≈2B scale, trained from scratch under matched data, seed, and GPU footprint; the hybrid offers fewer parameters and faster decoding at a modest training-throughput cost.

TABLE VII
ARCHITECTURE-LEVEL COMPARISON MOTIVATING THE HYBRID CHOICE (DENSE = MINICPM5-2B, HYBRID = BAIZE; MEASURED FROM SCRATCH UNDER MATCHED DATA, SEED, AND GPU FOOTPRINT, SAME NODE, SAME CONFIGURATION). TRAINING THROUGHPUT WAS RE-MEASURED ON A SINGLE 8×H100 NODE AFTER AN INITIAL CROSS-NODE MEASUREMENT WAS FOUND TO OVERSTATE THE GAP. THE DECODE GAP IS MEASURED UNDER THE MEGATRON-CORE DIRECT-DRIVE HARNESS WITHOUT CUDA GRAPHS, WHERE THE DENSE BASELINE IS CPU-LAUNCH-BOUND, AND THEREFORE NARROWS UNDER PRODUCTION INFERENCE STACKS (CF. TABLE XIV).

Metric	Dense	Hybrid	Edge
Parameters	2.51B	**2.22B**	hybrid (−11.6%)
Final loss (1k steps)	4.81	**4.68**	hybrid (−0.12)
Train loss (5k steps)	3.36	**3.06**	hybrid (−0.30)
Training tok/s	**90K**	83K	dense (+8%)
Prefill tok/s	21.4K	**23.7K**	hybrid (+10.5%)
Decode tok/s	2.2	**22.8**	hybrid (+10.3×)

J. Full-Scale Production Training

The search configuration of Section IV-I was scaled to the production budget of Table III: 2.94T tokens in the warmup-plus-stable phase, followed by the two-segment decay. The run uses the production configuration fixed by the throughput and precision studies: `TP`=1, `CP`=1, `DP`=16 over two 8×H100 nodes, micro-batch 1 at sequence length 16,384 (4,194,304 tokens/step at `GBS`=256), FP8 tensorwise with

delayed scaling, computing-graph replay disabled, AdamW (β_1=0.9, β_2=0.95), and the multi-layer tokenizer with an EOD token for document boundaries. Precision and replay are not incidental: at a micro-batch of M=16,384 the tensors are large enough that FP8 pays off, and replaying a graph with a data-dependent Mamba-2 state was the cause of a throughput collapse in an earlier build; both settings are fixed for the whole run so that the decay segments are comparable.

Stable phase (in progress). At the time of writing (31 October 2026) the run has passed iteration 30,660 of 700,000 (4.4%), at a sustained ≈598K tok/s (≈1,030 TFLOP/s/GPU; measured over 6.91–7.21 s/step). Training loss reached 2.151 (mean of the last five logged steps) from an initial ≈11.5, with gradient norms stable at 0.09–0.10 and no NaN or skipped step; the latest checkpoint is at iteration 30,000 (checkpoints are written every 2,000 iterations and pruned to a rolling window). The projection below is therefore an extrapolation at the measured rate, not a measurement.

TABLE VIII
PRODUCTION RUN ACCOUNTING. STABLE-PHASE WALL-CLOCK IS AN ESTIMATE AT THE MEASURED ≈590K TOK/S; DECAY FIGURES ARE ESTIMATES AT THE RUNG THROUGHPUTS OF TABLE XI; GPU-HOURS ARE NOMINAL (16 GPUS).

Phase	Tokens	Steps	Wall-clock	Status
Warmup + stable	2.94T	700k	≈56 d (est.)	30.7k done
Decay A (16K)	58.7B	14k	≈1.2 d (est.)	queued
Decay B (32K–128K)	25.2B	6k	≈2.3 d (est.)	queued, gated
Total	**3.02T**	**720k**	≈59.5 d	—

Decay—segment A (58.7B tokens, 16K). The learning rate begins its exponential decay while the sequence length stays at 16K. The blend shifts mass to the curated L3 and instruction-tuning sources (specified in `configs/blend_decay.tsv`): 60% general instruction tuning (`UltraData-SFT-2605`), 12% agentic instruction tuning (`UltraData-SFT-Agent-2609`), 18% curated web question–answer (`Ultra-FineWeb-L3-QA`), 4% Chinese web, 4% mathematics L3, and 2% code L3. Instruction data is used here as *text*: segment A is still pre-training, and the point of the shift is to expose the model to well-formed directive-response text at the lowest learning rates, not to teach the tool-call behaviour that Section V trains. Segment A anneals at 16K rather than at a longer length so that the model consolidates its short-context representations under a shrinking learning rate *before* the context distribution changes: entering segment B with an already-decayed learning rate means the length shift perturbs near-converged weights, not peak-LR weights. The 60–70% allocation protects short-context ability while the distribution changes, since the retrieval and long-document material of segment B is sampled from a different length regime and must not regress the capability that segment A anneals.

Decay—segment B (25.2B tokens, 32K–128K). Described in Section IV-K; it cannot start until the data conditions of Section IV-F are met.

K. Long-Context Training: The Decay Tail of the WSD Schedule

Positioning: a segment of the decay phase, not a separate stage. Long-context training is executed as segment B of the decay phase (Section IV-C), between the stable phase and post-training. It is not a standalone training stage, and it is placed neither in the stable phase nor in SFT/RL. Three properties of the schedule justify the placement. *(i) The learning rate is already low.* Segment B runs under the tail of the decayed learning rate (5.6×10^{-4} at p=0.25 falling to 1.8×10^{-4} at p=0.75; Table III), and a distribution shift applied at small step sizes perturbs general capability least; applied at the peak learning rate, the same shift would overwrite the short-context features that the stable phase spent 2.94T tokens acquiring. *(ii) SFT and RL do not establish ground retrieval ability.* SFT shapes instruction-following behaviour and RL optimises a policy preference over sampled outputs; neither exposes the model to the length-and-position distribution that retrieval depends on, and neither can be run at the token scale of segment B. *(iii) The extension must precede post-training.* Post-training assumes a backbone that can already address a long window; if the 1M context of the product target were only an extrapolation of a backbone trained to 16K, every post-training run would inherit that limitation. Table IX places segment B on the pipeline axis.

TABLE IX
STAGE (I) ON THE TRAINING AXIS. DECAY B (LONG-CONTEXT EXTENSION) IS PART OF THE DECAY PHASE AND PRECEDES POST-TRAINING.

	Warmup	Stable	Decay A	Decay B	SFT/RL
Seq. length	16K	16K	16K	32K–128K	16K+
Share	2k	2.94T	60–70%	30–40%	stage (ii)

Data budget. The decay phase is budgeted at 10–15% of the main training volume—294–441B tokens against the 2.94T of the stable phase—and long-context data takes 30–40% of that, i.e. 88–176B tokens (Table X). The ladder is not uniform: 40–50% is spent at 32K, 30% at 64K, and 20–30% at 128K, so that most long-context tokens are spent at the length where the model is currently weakest rather than at the headline length. We adopt 12% (353B) as the reference operating point at a 30% long-context share with a 12:9:6 rung split (42.4/31.8/21.2B; 95.4B in total), and segment B of Table VIII is its first rung-cycle. The remaining 60–70% of the decay phase stays at 16K on short-context data specifically to protect short-text ability, with a regression gate of $\leq$1 pp on CS8 and on complex reasoning (Table XII).

Composition inside the long-context segment. Synthetic needle-in-a-haystack data occupies 20–40% of the long-context tokens; natural long documents occupy the remainder. A purely synthetic segment is ruled out, because a model trained only on inserted needles learns to answer questions about inserted needles rather than to read long documents. The synthetic share is diversified over five protocols rather than

TABLE X
LONG-CONTEXT TOKEN BUDGET AS A FUNCTION OF THE DECAY-PHASE ALLOCATION. DECAY VOLUME IS A FRACTION OF THE 2.94T STABLE PHASE; RUNG SPLITS ARE 40–50% / 30% / 20–30% OVER 32K / 64K / 128K. ALL FIGURES ESTIMATED.

Decay	Decay tok.	LC	LC tok.	32K/64K/128K	16K
10%	294B	30–40%	88–118B	35/27/26B	206B
12% ref.	353B	30–40%	106–141B	42/32/32B	247B
15%	441B	30–40%	132–176B	53/40/40B	309B

one: single-needle NIAH; multi-needle MK-NIAH with distractor needles, where every needle must be returned for credit; multi-hop variable tracing; variable-length needles spanning one sentence to three paragraphs; and paraphrase-plus-noise variants in which the needle is reworded and the haystack is filled with topically related distractors. The natural share is drawn from the supplemented long-document sources of Section IV-F at building proportions of roughly 40% books and scientific articles, 20% repository-level code, and the remainder standards, legal, and documentation text; the haystack filler of the synthetic share is drawn from the same sources so that the two shares do not differ in style. Insertion depth is sampled uniformly over 0–100% instead of being concentrated at the middle, because the position sensitivity of a hybrid with four attention layers and 52 recurrent layers is not uniform along the window. **Training configuration.** The ladder runs 32K→64K→128K with context parallelism $\texttt{CP} \in \{2, 4, 8\}$ and $\texttt{DP} \in \{8, 4, 2\}$, holding $\texttt{seq}/\texttt{CP} = 16{,}384$ tokens per GPU at every rung, and global batch size is held constant within a rung (Table XI). The learning rate continues the decay and is never re-warmed: re-warming on entering a longer context would reintroduce the overwriting risk that motivated the placement in the first place. Position encoding uses adjusted base frequency from the first training step—$\theta{=}10^6$ is fixed at initialisation, so the decay tail carries no post-hoc frequency change, and 128K is reached by training rather than only by extrapolation, leaving 1M as an $8\times$ extrapolation. No position interpolation, NTK rescaling, or YaRN is enabled at any rung; a maximal-position change without the base-frequency change is excluded outright, because in our earlier sweep it produced NaN at every length tested. Document-level reset is mandatory: attention and the Mamba-2 recurrent state are reset at each EOD, so a window contains documents rather than document fragments.

TABLE XI
LONG-CONTEXT LADDER OF SEGMENT B. TOKENS PER GPU ARE HELD AT 16,384 AT EVERY RUNG. 32K THROUGHPUT IS MEASURED; 64K AND 128K ARE EXTRAPOLATED FROM MEASURED PER-GPU SCALING AND ARE MARKED AS ESTIMATES.

Rung	Seq.	CP / DP	GBS	Tokens	Throughput (tok/s)
1	32K	2 / 8	128	≈42B	351,526 (measured)
2	64K	4 / 4	64	≈32B	≈180K (est.)
3	128K	8 / 2	32	≈32B	≈90K (est.)

Evaluation checkpoint after the schedule. When the WSD schedule ends—that is, after both decay segments—we take one snapshot and measure *base* capabilities on it, reported separately from post-training results (Table XII): common-sense reasoning (CS8 and CX), needle retrieval (passkey and RULER, the 1M target being $\geq 95\%$), perplexity extrapolation ($1\text{M}/4\text{K} \leq 1.10\times$), and basic format following. Retrieval is scored on a depth-by-position grid with multi-needle all-hit scoring, because a checkpoint can extrapolate perplexity smoothly while failing retrieval outright; perplexity alone is not accepted as evidence of long-context ability. Complex reasoning and agentic ability are recorded at this snapshot as a baseline only; their improvement is the subject of Section V.

TABLE XII
POST-DECAY BASE-CAPABILITY SNAPSHOT (SINGLE EVALUATION POINT, BEFORE ANY POST-TRAINING). GATES ARE THE ACCEPTANCE CRITERIA FOR THE 1M PRODUCT TARGET.

Axis	Instrument	Gate
Common sense	CS8 (8-task), CX	$\leq$1 pp drop
Retrieval	passkey, NIAH/MK-NIAH at 1M	$\geq 95\%$
	RULER (mixed tasks)	$\geq 70\%$
PPL extrapolation	4K–1M curve	$\leq 1.1\times$
Format following	adherence suite	recorded
Complex reasoning	reasoning suite	baseline
Agentic	tool-use suite	baseline

Cost. At the measured 32K throughput and the extrapolated 64K/128K throughputs, the reference rung-cycle (42/32/32B) costs ≈118 GPU-hours on sixteen GPUs, of which the 32K rung alone is 33 hours; the upper bound of Table X (15%/40%) costs ≈220 GPU-hours. The cost is carried by context parallelism rather than by additional nodes: 32K runs at 59% of the 16K throughput, and that gap is the price of the training signal, not of the hardware.

Why the budget is this small. The long-context budget is deliberately an order of magnitude below the stable-phase budget, and the public precedents bound it from both sides. ProLong-8B reaches a 128K context with state-of-the-art performance at its size after 40B tokens of long-context training, which is already only 5% of the long-context tokens used by Llama-3.1-8B-Instruct [17]; LongRoPE2 extends LLaMA3-8B to a 128K effective context with 10B tokens while retaining 98.5% of short-context performance [18]. Our 88–176B long-context budget (Table X) therefore sits above the public operating range, which is the conservative direction for a plan whose data gate (Section IV-F) is not yet satisfied. Two qualifications keep the comparison honest: both precedents extend an *already long-trained* base model by rescaling position encoding, whereas segment B trains the length structure itself under a decayed learning rate; and neither operates at the 2.22B scale, where the token requirement is not expected to exceed the 8B-class figures above but is not measured here either. Long-context ability is accordingly not bought with volume—it is bought with the right volume at the right point in the schedule—but the reference budget in Table X should be read as a floor that the data gate may push upward, not as a ceiling (estimate; no scale ablation of this axis is run in this

paper).

L. Long-Context Capability and Inference Cost

The hybrid backbone's long-context behaviour was characterised along two axes: perplexity extrapolation and inference throughput under a production serving stack.

Perplexity extrapolation. With adjusted base frequency (ABF, $\theta_{\text{rope}}=10^6$) applied to the four attention layers, the 20K-step (≈655M-token) checkpoint sustains near-constant perplexity from 4K to 1M tokens without further training: PPL ranges from 51.5 (64K) to 55.4 (1M), a change of only $+7.6\%$ over a $256\times$ context increase (Table XIII). This stability arises because only 4 of 56 layers carry RoPE position encoding; the 52 Mamba-2 state-space layers have no position dependency and therefore incur no out-of-distribution penalty at long range.

TABLE XIII
TRAINING-FREE LONG-CONTEXT PERPLEXITY OF THE MAMBA2-HYBRID (20K-STEP, ≈655M-TOKEN CHECKPOINT, ABF $\theta=10^6$, $b=1$).

Context	PPL	Prefill (s)	Peak VRAM
4K	83.2†	—	—
64K	51.5	—	—
128K	53.3	10.7	10.9 GB
256K	53.0	13.8	14.3 GB
512K	55.8	53.0	19.7 GB
1M	55.4	172.4	34.7 GB

†The 4K PPL (83.2) is measured on a held-out evaluation shard with a different data distribution and loss-aggregation method than the training-loss reported in Section IV-I; the two are not directly comparable. The key observation is the *stability* from 64K to 1M (51.5→55.4, $+7.6\%$), not the absolute 4K value.

Inference throughput. Under the SGLang production serving stack (CUDA graphs, PagedAttention, FlashInfer backend) with warm-up, the hybrid outperforms the dense baseline at long context: at 128K the hybrid achieves $2.8\times$ higher prefill throughput and $1.6\times$ higher decode throughput; at 256K the gaps widen to $4.3\times$ and $1.9\times$ respectively (Table XIV). The dense model cannot serve beyond 512K on a single 80 GB H100 (KV-cache pool cap ≈455K tokens at 30% memory fraction), whereas the hybrid serves through 1M with near-constant ≈26 GB VRAM, as the Mamba-2 state grows in $O(1)$ for 52 of 56 layers.

TABLE XIV
LONG-CONTEXT INFERENCE COST COMPARISON (SGLANG, WARM-UP CORRECTED, $b=1$, GEN_LEN=64, 80 GB H100). DENSE ENTRIES MARKED "—" ARE OUT-OF-MEMORY.

	Prefill (tok/s)		Decode (tok/s)	
Ctx	Hybrid	Dense	Hybrid	Dense
128K	39 377	14 051	311	193
256K	32 381	7 532	276	144
512K	23 634	—	220	—
1M	15 530	—	159	—

Production training throughput. Two measurements fix the production configuration. On a single $8\times$H100 node, the hybrid sustains 249 040 tok/s at TP=1/DP=8, micro-batch 2, sequence 4096 in BF16 (≈600 TFLOP/s/GPU). At the production length the picture changes: with sequence 16,384 and CP=1/DP=16 across two nodes, FP8 tensorwise with delayed scaling sustains 613,992 tok/s, and the sustained rate observed on the live run is ≈590K tok/s over 7.08–7.32 s per 4.19M-token step. Computing-graph replay is disabled throughout: a replayed graph over a data-dependent Mamba-2 state ran into a throughput plateau, and disabling replay is a precondition for the rate above.

Precision choice. At the 4096-token screen the FP8 tile ($M=8192$) sits below the crossover point (M^*≈30–32K) and ran at $0.91\times$ the BF16 throughput, so the screen retained BF16; a 1000-step A/B at $M=65{,}536$ additionally showed no persistent loss divergence between BF16 and delayed FP8 (0.19% end-of-run difference, one transient spike at step 700 recovering within ∼60 steps). At $M=16{,}384$ the tile is large enough for FP8 to pay, and the production run adopts FP8 tensorwise: the 613,992 tok/s figure is an FP8 measurement, and the BF16 rate at the same shape is not directly measured. The two results are therefore consistent rather than contradictory—BF16 is the right choice at short sequence length, FP8 at production length—and the decision was made per configuration, not globally.

Table XV reports the hyper-parameter search outcomes at a matched horizon; the searched axes target the hybrid training recipe (stable learning rate, warmup length, decay ratio, and schedule family).

TABLE XV
HYPER-PARAMETER AND DATA-MIX SEARCH OUTCOMES AT A MATCHED 5000-STEP (S5: 20000-STEP) HORIZON.

ID	Change vs. best	Val. loss	Outcome
S1-01	baseline ($lr=3\times10^{-4}$, WSD)	2.849	baseline
S2-07	stable LR $= 1 \times 10^{-3}$	**2.7627**	best LR
S3-02	schedule family: cosine	2.8155	WSD retains
S3-03	decay ratio 5%	2.7900	10% retains
S3-04	warmup 2000 steps	2.7957	250 retains
S4-03	anneal L3+code+math 86:10:4	**2.6298**	best mix
S5-01	winner, 20000 steps	**2.2054**	**winner**

Note: The search-stage IDs (S1–S5) are re-numbered for presentation; they correspond to the internal experiment ledger as follows: S1-01→S2-02, S2-07→S2-07, S3-02/03/04→S3-02/03/04, S4-03→S4-03, S5-01→S5-01.

V. LLM POST-TRAINING

Stage (ii) adapts the pre-trained backbone to EDA scripting, and it is the stage that carries BaiZe's domain claim: the pre-training corpus of Section IV-E is general web, code, and mathematics text and contains no EDA artifacts, so the domain prior must be introduced here. We organise the stage as three steps—*domain annealing*, *supervised fine-tuning*, and *reinforcement learning*—all evaluated under a single frozen protocol (Section V-D).

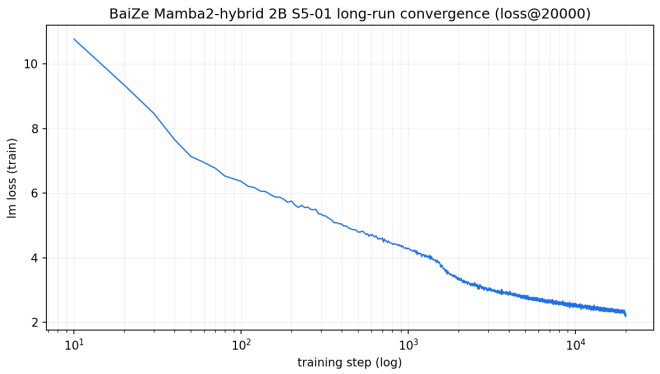


Fig. 1. Training-loss curve of the winning Mamba2-hybrid configuration over the 20000-step verification run (seed 1234); the decay tail (steps 19500–20000) anneals the loss from ~ 2.34 to 2.2054.

A. Domain Annealing

The cheapest lever on the domain prior is to continue pre-training on EDA text before any behavioural tuning. We reuse the warmup–stable–decay machinery of Stage (i) and anneal on a corpus assembled from PyAether and SKILL API references, EDA tool documentation, and open HDL sources, blended with the general mixture at a controlled ratio. Because the step changes only the data distribution, we contrast it against an SFT-only control to isolate its contribution from the behavioural tuning that follows.

B. Supervised Fine-Tuning

SFT teaches the generate–execute–refine loop that the deployment target requires. The corpus combines the agentic instruction set (`UltraData-SFT-Agent-2609`, 51 G across 50 shards) and a general instruction set (`UltraData-SFT-2605`, 319 G across 1,504 JSONL files, verified byte-identical to the HuggingFace manifest) with PyAether-specific material synthesised from API documentation. We ablate the data mixture (agentic / code / mathematics / PyAether), the presence of interleaved tool-call traces, full fine-tuning versus LoRA, and the sequence length: agentic traces—code, tool returns, and successive refinements—routinely exceed the 4096-token window of Table I, so window extension is treated as a first-class variable rather than a fixed setting.

C. Reinforcement Learning with Verifiable Rewards

An execution sandbox turns correctness into a verifiable signal, so we optimise with group-relative policy optimisation and no value network. Table XVI separates the reward variants we study. A binary signal is the simplest baseline; partial credit over executable attempts addresses the sparsity of pass/fail; a tool-call penalty aligns the policy with the harness cost model; and an API-legality term penalises hallucinated symbols. Because the reward is computed from assertions a policy could in principle target directly, we keep the harness anti-cheat hook enabled in every run and report a cheat rate beside every accuracy figure.

TABLE XVI
REWARD VARIANTS STUDIED IN STAGE (II).

ID	Reward
R1	binary pass/fail
R2	executable + assertion pass ratio
R3	R2 + tool-call penalty
R4	R3 + API-legality term

D. Evaluation Protocol

We track four capability milestones through Stage (ii): **M1** tool-call legality (syntactically valid PyAether calls, no hallucinated symbols); **M2** single-task pass (Pass@1 on EDA-Eval-PyAether); **M3** multi-turn refinement (recovery from execution failures with up to 3 rounds); **M4** full-benchmark accuracy. All variants are scored on **EDA-Eval-PyAether** [19] (158 real-world EDA scripting tasks), each pairing a natural-language prompt and function signature with assertion-based test code, scored by execution in the vendor sandbox. We follow the benchmark's own protocol—five independent runs, reshuffled order, fresh context, mean Pass@1 with run-to-run standard deviation, 2500-s per-task timeout—so numbers are comparable to published ones. Two controls make the comparison interpretable: contamination is excluded by the n-gram blacklist of Section IV-E, and only the backbone is varied while the retrieval stack, sandbox, anti-cheat hook, and agent scaffold are held byte-identical, since the harness supplies most of the observed accuracy.

Results are reported as **[TBD]**.

VI. Vision Encoder Pretraining

Stage (iii) pre-trains a custom vision encoder from scratch (Section VII); we summarise the outcome here and defer the design study to a companion report.

a) Selected encoder.: We adopt **OpenVision2**, a width-1024/depth-30 pure-attention ViT with 126.8M parameters (505.2M at full width), trained from scratch with a bidirectional trunk at 224px/16-px patches on a ≈18.5M-pair image–text corpus (CC12M + Amshaker). The objective combines a global InfoNCE contrastive term with a caption-*independent* masked-patch reconstruction term at a 1:1 weight and a 0.6 mask ratio. Both terms are required: the dense term carries most of the representation quality, while the contrastive term acts as an anchor without which the trunk degrades.

b) Final result.: On the frozen trunk with an ImageNet-1k linear probe, the encoder reaches $11.50\% \rightarrow \mathbf{19.76}\%$ top-1 from 5.1M to 55.3M samples, and 20.27% at 176M samples with no saturation. This is $+12.4$ pp above a contrastive-only baseline at the same sample budget, and it is the operating point used by stage (iv). The $224/16$ resolution is retained because it is best on both loss and throughput; whether 196 visual tokens suffice for detail-dense layout and schematic views is tested in stage (iv) (Section VII).

VII. MLLM Alignment

Stage (iv) connects the vision encoder of Section VI to the language backbone through a lightweight projector, following the staged curriculum established for open multimodal assistants: projector pre-training, mid-training, and instruction tuning. The design is motivated by the observation that the dominant EDA artifacts are visual—layout and schematic views account for 70.2% of the tasks in the evaluation benchmark [19]—so a multimodal core can be scored on the same execution-based protocol as the text-only one.

a) Task construction.: We build three task families, each chosen because it admits an objective ground truth. (i) *image→script*: given a rendered layout or schematic view, produce the PyAether or SKILL script that recreates it; supervision is available because executing a script yields its own reference rendering. (ii) *script→image*: predict the geometry that a given script produces, again self-labelled by execution. (iii) *layout VQA*: answer attribute questions—layer identity, device type, spacing and rule violations—labelled by domain experts. The first family is the strongest of the three: its output is a script, so it is scored by the same assertion sandbox used in stage (ii), which gives the multimodal and text-only stages a common metric instead of two incommensurable ones.

b) Curriculum and ablations.: We run stage-1 projector pre-training on caption-style pairs, stage-1.5 mid-training on the three families above, and stage-2 instruction tuning on the full mixture. The ablations vary the projector (MLP versus query-based), whether the vision tower is frozen or updated, and the number of visual tokens per image. The last is of particular interest. The architecture study of Section VI selected $224/16$, where an image becomes only $14\times14 = 196$ tokens ($\sim16\times16$ px/token)—a regime in which the training objective is dominated by coarse semantics. For detail-dense line drawings, a 336-px view would yield 576 tokens ($2.9\times$ more) and a 448-px view 1024 tokens ($5.2\times$ more); whether the 196-token setting transfers to such views is precisely what stage (iv) tests. Layout and schematic views are the opposite: they are detail-dense line drawings whose meaning lives in fine geometry. We therefore treat resolution and visual-token count as first-order variables here rather than inheriting the earlier null result, which we read as a property of the proxy task and not of the encoder.

Results are reported as **[TBD]**.

VIII. Semiconductor Domain Adaptation

Stage (v) completes the pipeline by adapting the multimodal model to proprietary design practice. It differs from stage (ii) in two respects, which we state explicitly because otherwise the two stages would coincide: the reward is no longer verifiable but *learned from human judgement*, and the target behaviour is no longer general API usage but a team's internal conventions—naming rules, layer policies, review checklists—which no public benchmark measures.

a) Method.: We collect pairwise judgements from in-house design reviews over the stage-(iv) task families, fit a domain reward model on them, and optimise the aligned model against that model under a KL-regularised objective. Because preference data is far scarcer than the verifiable signal of stage (ii), we initialise from the stage-(iv) checkpoint and constrain the update rather than training the objective from the pre-trained model.

b) Why a small model.: The rationale for a ≈2.2B core is strongest here, and it is a constraint rather than a preference. Proprietary layouts and proprietary preference data cannot leave the premises, so the choice is not between a large hosted model and a small local one—it is between a small local model and no model at all. Serving cost then dominates, and for an agentic session the relevant cost is not throughput on a short prompt but memory that grows with accumulated context: in the hybrid backbone of Section III-A, the four attention layers are the only component whose serving state grows with the session, while the Mamba-2 layers hold a constant-size recurrent state.

c) Evaluation.: We evaluate on held-out review items using expert agreement, and report the result beside the execution-based numbers of stage (ii) so that verifiable and preference-based adaptation can be compared on the same system rather than in separate accounts.

Results are reported as **[TBD]**.

IX. Conclusion

We presented BaiZe, an end-to-end multimodal agentic system: a ≈2.2B-parameter hybrid (state-space + attention) language model built from scratch, post-trained for agentic capability, paired with a custom vision encoder, aligned into a multimodal model, and adapted to the semiconductor domain. Through a controlled from-scratch comparison, we selected a sequential Mamba2-hybrid backbone over a dense transformer for its lower loss-per-parameter and substantially faster inference, then applied a staged, budget-constrained hyperparameter search (S1–S5) on the final backbone. Stages (ii) to (v) are specified here as a protocol and evaluated on a common execution-based benchmark [19]; their measurements are reported as **[TBD]**. The complete recipe and a single reproducible command encapsulate the entire construction under the NVIDIA NeMo recipe.

The immediate next step is agentic fine-tuning with the `UltraData-SFT-Agent` corpus toward a deployable agent core for EDA and embedded applications. We hope this blueprint lowers the barrier for cost-bounded teams building their own efficient agentic SLMs from scratch.

References

[1] S. Hu, Y. Tu, X. Han, C. He, G. Cui, X. Long, Z. Zheng, Y. Fang, Y. Huang, W. Zhao *et al.*, "Minicpm: Unveiling the potential of small language models with scalable training strategies," *arXiv preprint arXiv:2404.06395*, 2024.

[2] A. Yang, B. Li, B. Zhang *et al.*, "Qwen3 technical report," 2025.

[3] G. Team, "Gemma 3 technical report," 2025.

[4] X. Liu, H. Yu, H. Zhang, Y. Xu, X. Lei, H. Lai, Y. Gu, H. Ding, K. Men, K. Yang *et al.*, "Agentbench: Evaluating llms as agents," in *International Conference on Learning Representations (ICLR)*, 2024.

[5] A. Gu and T. Dao, "Mamba: Linear-time sequence modeling with selective state spaces," *arXiv preprint arXiv:2312.00752*, 2023.

[6] T. Dao and A. Gu, "Transformers are ssms: Generalized models and efficient algorithms through structured state space duality," *arXiv preprint arXiv:2405.21060*, 2024.

[7] NVIDIA, "Nemotron-h: A family of accurate and efficient hybrid mamba-transformer models," 2025.

[8] X. Dong, Y. Fu, S. Diao *et al.*, "Hymba: A hybrid-head architecture for small language models," 2024.

[9] H. Liu, C. Li, Q. Wu, and Y. J. Lee, "Visual instruction tuning," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[10] B. Li, Y. Zhang, D. Guo, R. Zhang, F. Li, H. Zhang, K. Zhang, P. Zhang, Y. Li, Z. Liu, and C. Li, "Llava-onevision: Easy visual task transfer," 2024.

[11] M. Liu, T. E. Ene, R. Kirby, C. Cheng, N. Pinckney, R. Liang, J. Alben, H. Anand, S. Banerjee, I. Bayraktaroglu *et al.*, "Chipnemo: Domain-adapted llms for chip design," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[12] J. Blocklove, R. Chapman, B. Tan, D. Chen *et al.*, "Chipbench: The llm chip design benchmark," 2024.

[13] OpenBMB, "Minicpm5: Openbmb small language model series," https://github.com/OpenBMB/MiniCPM, 2026, model release (1B: 2026-05, 2B: 2026-09); no independent training paper available at time of writing.

[14] NVIDIA, "Nemo: A toolkit for building ai applications using neural modules," https://github.com/NVIDIA/NeMo, 2024, version 2.x; used for pre-training via the NeMo run entry point.

[15] M. Shoeybi, M. Patwary, R. Puri, P. LeGresley, J. Casper, and B. Catanzaro, "Megatron-lm: Training multi-billion parameter language models using model parallelism," 2020.

[16] DeepSeek-AI, "Deepseek-v3 technical report," 2024, tokenizer (DeepSeek-V4.1-Flash variant) reused for BaiZe; vocabulary size 129,281 after EOD append.

[17] T. Gao, A. Wettig, H. Yen, and D. Chen, "How to train long-context language models (effectively)," in *Annual Meeting of the Association for Computational Linguistics (ACL)*, 2025, proLong-8B: long-context training on 40B tokens reaches 128K SOTA, using 5% as many long-context tokens as Llama-3.1-8B-Instruct.

[18] N. Shang, L. L. Zhang, S. Wang, G. Zhang, G. Lopez, F. Yang, W. Chen, and M. Yang, "Longrope2: Near-lossless llm context window scaling," 2025, extends LLaMA3-8B to 128K effective context using only 10B tokens while retaining 98.5% of short-context performance.

[19] Y. Liu, S. Hou, X. Chen, Y. Wang, S. Yuan, Q. Gan, S. You, F. Chen, W. Li, S. Hu, Y. Liu, E. Xia, X. Lu, H. Wang, F. Xu, and Y. Li, "Zhulong: Execution-grounded llm agent for eda scripting with offline api self-exploration," 2026.